SC2107 Lab1 Assignment Sheet (to be submitted to NTULearn before next lab)

Name: Jothilingam Dheeraj Lab Group: SCE3 Date: 19/09/24

1. Section 7.4. Write one C statement to set bit 7 and 5 of P1SEL0 register, keeping the rest of the bits in the register unchanged.

   To modify specific bits within a register while leaving the rest untouched, a **BITWISE OR** operation is typically used. The C code to achieve this would be:

   P1SEL0 |= (1 << 7) | (1 << 5);

2. Section 7.4. Write C statement(s) to extract bit 6 and 5 of variable 'x' and right align these two bits. Masked off all other bits in variable 'x'. e.g. if 'x' has a value 1101 0111b initially, it should have a value of 0000 0010b after executing the C statement.

   To extract and reposition bits 6 and 5 of a variable, we start by applying a mask with the value **0x60** (which corresponds to 0110 0000b in binary). After masking, we shift the bits 5 positions to the right so that they are right aligned. The C code to perform this task is:

   x = (x & 0x60) >> 5;

3. Section 7.4. Why do we need to declare the P1IN register, the register that contain the status of the processor Port1 GPIO input pin logic with a 'volatile' keyword qualifier?

   The **`volatile`** keyword is necessary because without it, the compiler might optimize away operations involving the **`P1IN`** register, assuming its value won't change. However, since **`P1IN`** reflects the state of external GPIO pins, its value can be altered outside the program by hardware or interrupts.

   By declaring it as **`volatile`**, we instruct the compiler to always access the actual memory location of **`P1IN`**, ensuring that the latest value from the input pins is used, even if the change happens asynchronously.

4. Section 7.5. Why do we use SDIV instead of UDIV when calculating the Distance D? Or does it really matter whether SDIV or UDIV is used for this case?

   The choice between **SDIV** (signed division) and **UDIV** (unsigned division) depends on whether the numbers being divided are signed or unsigned. In the expression $D=1195172/n-1058$, we opt for SDIV because the result of $n-1058$ might occasionally be negative. Although `n` is an unsigned value, subtracting 1058 could result in a negative outcome. Using UDIV for a negative result would lead to incorrect behaviour, as UDIV would treat the negative number as a large positive one due to underflow.

   Even though there's a **saturation** mechanism preventing `n` from going below 2552, SDIV is generally preferred here to ensure accuracy. The subtraction step makes SDIV the safer option, even if UDIV could theoretically work in most cases.

5. Section 7.5. What is saved into the LR register when the calling routine calls “BL Convert”? What command is used to return from the sub-routine to the calling routine?

   During the execution of the **BL Convert** instruction, the **Link Register (LR)** holds the **return address**—the location where the program needs to continue after the subroutine completes. This return address allows the function to jump back to the correct point after execution.
   To return from the subroutine to the calling function, the command **BX LR** is executed, which uses the value stored in the LR to direct the program back to its original execution flow.

6. Section 7.5. If a function has 4 input parameters, which registers does the calling routine used to pass these parameters to the function according to AAPCS?

   According to the **ARM Architecture Procedure Call Standard (AAPCS)**, the first four input parameters are passed using the following registers:

- **R0:** First parameter
- **R1:** Second parameter
- **R2:** Third parameter
- **R3:** Fourth parameter

   If there are more than four parameters, the additional ones are passed using the **stack**.

7. Section 7.6. What data content is loaded into R1 by the instruction “ldr r1, [pc, #0x2e4]”? Just the expression will do, need not give the exact value since the offset in your code may be different.

   The instruction **ldr r1, [pc, #0x2e4**] calculates the address by adding the offset **0x2E4** to the current value of the **Program Counter (PC)**, which points to the instruction currently being executed. The value at this computed address is then loaded into the **R1** register.
   Thus, the expression effectively loads the value located at **PC + 0x2E4** into **R1**.

8. Section 7.7. The Memory Section “MAIN” correspond to the On-Chip Flash Memory in MSP432. How much on-chip flash memory is available for future code development? Cut and paste the screen shot of the relevant content in the map file and highlight where you extract your answer from. Hint: Check the map file.

```
MEMORY CONFIGURATION

        name             origin    length      used     unused   attr    fill
----------------------   --------  ---------  --------  --------  ----  --------
  MAIN                   00000000  00040000  000007fc  0003f804  R  X
  INFO                   00200000  00004000  00000000  00004000  R  X
  SRAM_CODE              01000000  00010000  0000021c  0000fde4  RW X
  SRAM_DATA              20000000  00010000  0000021c  0000fde4  RW
```

To determine how much flash memory is available for future use, we examine the **MAIN** section, which represents the On-Chip Flash Memory.

- The total length of the **MAIN** section is 0x00040000.
- So far, 0x000007fc of this space has been utilized, leaving 0x0003f804 of unused memory.

  To calculate this in bytes:
- 0x3f804 in hexadecimal can be converted to decimal:

  3×16^4+15×16^3+8×16^2+0×16+4=260,100 bytes.

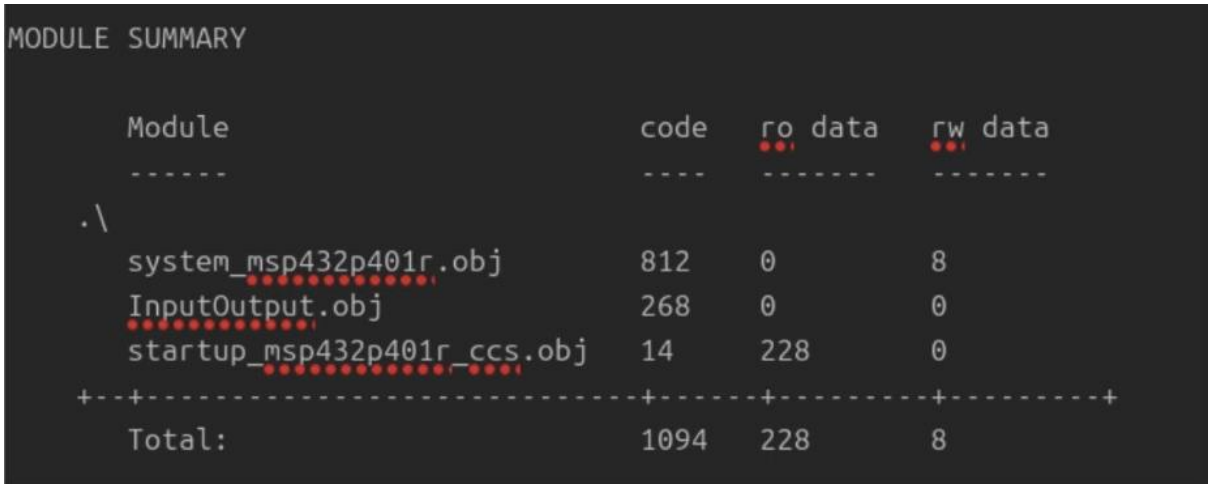
  Thus, **260,100 bytes** of on-chip flash memory are still available for future code development.

9. Section 7.7. Which software section are code allocated to by default? Which file consumes the largest code size in this project? Hint: check the map file.

By default, code is allocated to the **.text** section, as indicated in the Section Allocation Map.

```
.text      0    000000e4   000006f0
                000000e4   0000032c   system_msp432p401r.obj (.text)
                00000410   0000010c   InputOutput.obj (.text)
                0000051c   0000009c   rtsv7M4_T_le_v4SPD16_eabi.lib : memcpy_t2.obj (.text)
                000005b8   00000070                                : autoinit.obj (.text)
                00000628   00000068                                : copy_decompress_lzss.obj (.text:decompress:lzss)
                00000690   00000054                                : boot.obj (.text)
                000006e4   00000054                                : exit.obj (.text)
                00000738   0000004c                                : cpy_tbl.obj (.text)
                00000784   00000018                                : args_main.obj (.text)
                0000079c   00000014                                : _lock.obj (.text)
                000007b0   0000000e                                : copy_decompress_none.obj (.text:decompress:none)
                000007be   0000000e   startup_msp432p401r_ccs.obj (.text)
                000007cc   00000004   rtsv7M4_T_le_v4SPD16_eabi.lib : pre_init.obj (.text)
                000007d0   00000002                                : mpu_init.obj (.text)
                000007d2   00000002                                : startup.obj (.text)
```

In this project, the file **system_msp432p401r.obj** takes up the most memory, using **812 bytes**. The next largest is **InputOutput.obj**, which uses **268 bytes**. This information can be seen in the Module Summary section of the map file.



```
MODULE SUMMARY

      Module                       code   ro data   rw data
      ------                       ----   -------   -------
   .\
      system_msp432p401r.obj       812    0         8
      InputOutput.obj              268    0         0
      startup_msp432p401r_ccs.obj  14     228       0
   +--+----------------------------+------+---------+---------+
      Total:                       1094   228       8
```

10. Section 7.7. From the map file, what is the starting address of Port2_Init()? Compare with the address you see in the Disassembly Window, are they the same? If not, why?

The starting address of Port2_Init() from the **Global Symbols** section of the map file is **0x0000043b**.



```
0000045b  Port1_Output
0000043b  Port2_Init
00000479  Port2_Output
```

However, the address shown in the Disassembly Window may differ. This difference can be attributed to several reasons:

First is code optimization & inlining**.** The compiler may optimize the code by embedding certain functions directly into the calling functions, rather than allocating them specific memory addresses. This could shift the function's actual memory location during runtime.

Second is linker script memory allocation**.** The linker script controls where different parts of the code and data are placed in memory. During execution, addresses shown in the disassembly window may vary due to factors such as bootloader sequences or interrupts.

Third is address alignment**.** Compilers may align certain sections of the code to optimize memory access times or comply with hardware specifications.

Additionally, other factors like dynamic relocation or flash remapping might contribute to the difference.